EMPLOYEE LEAVE MANAGEMENT

SPECIFICATION-DRIVEN DEVELOPMENT (SDD)

Employee leave request, approval and balance tracking

Version 1.0 | 29 October 2026 | Draft for stakeholder review

Layer	Technology
Frontend	React JS
API	FastAPI (Python)
Database	PostgreSQL
Status	Draft — proposed requirements require stakeholder review

Purpose and intended outcome

Build a web application that lets employees apply for leave, view leave balances and track request decisions. Managers review requests from their direct reports. HR administrators configure leave types and maintain policy-controlled balances. This specification defines observable behavior and verification criteria for an initial release suitable for implementation planning and testing.

How to read this specification

S = core requirement in this specification. P = proposed elaboration needed to make the brief implementable and testable. Proposed rules are a draft baseline, not confirmed company policy. “Shall” describes required behavior once the baseline is adopted. Requirement IDs connect implementation work to acceptance criteria and tests.

Document map

1. Scope and baseline assumptions
2. Actors and access control
3. Functional requirements
4. Leave lifecycle and business rules
5. Web interface specification
6. Data model and validation
7. REST API contract
8. Errors, notifications and reliability
9. Quality and operational requirements
10. Acceptance criteria and test cases
11. Delivery plan and decisions

1. Scope and baseline assumptions

Purpose

Build a web-based Employee Leave Management System (ELMS) where employees can request leave, view balances and track decisions. Managers review requests from their direct reports, while HR administrators maintain leave types, policies and employee records. The system exposes a React JS web UI, FastAPI REST API and PostgreSQL database.

Included in the initial release

Authentication and role-based access; employee profile view; leave type and balance view; leave application, cancellation request and history; manager approval/rejection; HR administration of leave types and balances; in-app notifications; audit history; filtering and pagination; API validation and automated tests.

Core requirements

S01 Employees shall submit leave requests. S02 Employees shall view leave balances and request status. S03 Managers shall approve or reject requests assigned to them. S04 HR shall manage leave types and balances. S05 The system shall validate dates, required fields, balances and permissions. S06 The system shall persist requests and expose REST APIs. S07 A newly submitted request shall start in Pending status. S08 Verification shall include valid, invalid and unauthorized actions.

Proposed assumptions

A01 One organization is supported in the first release. A02 Roles are Employee, Manager and HR Admin; a user may have one primary role, with manager relationships stored separately. A03 A manager can decide requests for their assigned direct reports only; HR Admin can view and administer organization-wide leave data. A04 Leave types, annual entitlements, carry-forward, holidays, half-days and accrual rules are configurable policy decisions. A simple configurable balance model is used for the MVP. A05 Dates are interpreted in the organization’s configured timezone. A06 In-app notifications are in scope; email/SMS/push delivery is out of scope unless approved.

Excluded from this release

Payroll integration, biometric attendance, shift scheduling, travel requests, expense claims, automatic HRIS synchronization, mobile native apps, advanced accrual engine, multi-country policy engine and external email/SMS delivery. These require separate requirements and approval.

2. Actors and access control

Actors and permissions

Employee: view own profile, balances and requests; create requests; view own request history; request cancellation of eligible requests. Manager: all employee capabilities for self, plus view requests from direct reports and approve/reject those requests. HR Admin: manage leave types, policy configuration and balances; view organization-wide requests; correct records through audited administrative actions.

Authorization rules

P01 Every protected page and API endpoint requires an authenticated active account. P02 The server derives user identity and role from a validated session/token; client-supplied user IDs or roles never establish access. P03 Employees may access only their own requests and balances. P04 Managers may decide only requests belonging to their direct reports, unless an explicitly approved delegation rule is implemented. P05 HR Admin access is organization-wide and all administrative changes are audited. P06 Inaccessible and nonexistent request IDs return the same 404 response to avoid data leakage. P07 Inactive users cannot sign in or be assigned as an active manager.

Authentication baseline

Use the organization’s identity provider where available. For a local development setup, use a documented authentication mechanism and hashed passwords only; never store plaintext passwords. Production cookie authentication must use Secure, HttpOnly and appropriate SameSite settings plus CSRF protection for writes.

3. Functional requirements

Submit and view leave

FR01 The employee submits leave type, start date, end date and reason; optional half-day details are accepted only if enabled by policy. The server creates a unique request ID, records the authenticated employee and sets status to Pending. FR02 Required fields, date order, policy limits, overlapping requests and balance rules are validated server-side. Invalid submissions create no request or audit event. FR03 Employees see their own requests with leave type, dates, requested days, status, submitted time and decision details. Lists support status/date filters, search where relevant and pagination.

Manager decisions

FR04 Managers see pending and historical requests for direct reports. FR05 A manager may approve or reject a Pending request; rejection requires a meaningful comment. FR06 A manager cannot approve their own request unless a separately approved delegation policy explicitly permits it. FR07 Approval checks the latest request state and leave balance within a database transaction. FR08 Requests that are already decided cannot be decided again; stale/conflicting decisions return a conflict response.

Balances and HR administration

FR09 The system displays each employee's balance by leave type, including entitled, used, pending/reserved and available days where supported by the chosen policy. FR10 HR Admin can create/deactivate leave types and configure whether a type is paid, balance-limited and available for partial-day requests. FR11 HR Admin can adjust balances only with a reason; adjustments are recorded in immutable audit history. FR12 Deactivation of a leave type does not remove historical requests.

Cancellation, history and notifications

FR13 Employees may request cancellation of Pending requests; the request is cancelled only after the server confirms the request is still Pending. Cancellation of approved leave is a proposed policy decision and should require manager/HR action if enabled. FR14 Every submission, decision, cancellation and HR adjustment creates an audit event with actor and server timestamp. FR15 In-app notifications are created for submission confirmation and manager decisions. Notification summaries avoid exposing sensitive reason text. FR16 Viewing a notification does not mark it read; users explicitly mark their own notifications as read.

4. Leave lifecycle and business rules

Status lifecycle

Draft (client-only; not persisted in MVP) → Pending on successful submission. Pending → Approved or Rejected by an authorized manager. Pending → Cancelled by the employee if policy permits. Approved → Cancellation Requested → Cancelled may be enabled by policy; if not enabled, approved requests cannot be cancelled through the employee UI. Rejected, Cancelled and Approved requests are terminal for the basic MVP unless an audited HR correction process is defined.

Date and balance rules

P08 End date must be on or after start date. P09 The server calculates requested working days using the configured workweek and holiday calendar; do not trust a client-supplied day count. P10 Overlapping Pending or Approved requests for the same employee are rejected unless policy explicitly allows overlap. P11 Balance-limited leave cannot be approved if the available balance is insufficient. P12 Pending requests may be shown as reserved balance, but the organization must choose whether reservation reduces the displayed available balance. P13 Rejected and cancelled requests do not consume leave balance. P14 Timezone, half-day, weekends, public holidays, carry-forward, negative balances and retroactive requests require HR policy confirmation.

Atomicity and concurrency

P15 A decision, balance update, audit event and notification shall commit atomically. P16 The API shall use a request version or conditional update to prevent two managers from deciding the same request concurrently. P17 Repeated identical submissions should not create duplicate requests when the same idempotency key is retried. P18 Only actual changes generate audit events and notifications.

5. Web interface specification

Required screens

Sign in: organizational authentication and return to intended page. Employee Dashboard: balance summary, pending requests and recent activity. Apply for Leave: leave type, date range, reason and policy guidance. My Leave Requests: searchable/filterable request list with status labels and details. Request Details: dates, calculated days, status timeline, decision comment and audit-visible history appropriate to the role. Manager Inbox: pending requests from direct reports with approve/reject actions and rejection comment. Team Calendar: optional simple list/calendar view of approved absences if included in MVP. HR Administration: manage leave types, balance adjustments and employee-manager relationships where permitted. Notifications: unread indicator and mark-as-read action.

Form and UI behavior

UI01 Required fields are labelled and whitespace-only values fail validation. UI02 Client validation mirrors server rules, but the server remains authoritative. UI03 Preserve form entries after validation or network errors. UI04 Do not show success until the API confirms the write. UI05 Display status as text as well as color; show clear loading, empty and error states. UI06 On 401 redirect to sign-in; on 403 explain the action is unavailable; on 404 show a generic unavailable message; on 409 ask the user to refresh and review current state. UI07 Support keyboard navigation, visible focus, associated labels and responsive layouts at 375px and 1440px widths. UI08 Display dates in the organization/user timezone with the timezone identified.

React implementation boundary

React JS owns presentation, client-side form state, navigation and API calls. It must not implement authoritative permissions, balance calculations or approval decisions. Shared API calls should be placed in a small service layer; route guards improve UX but do not replace FastAPI authorization.

6. Data model and validation

Core entities

User: id UUID, external_subject or login identifier, display_name, email, role, manager_id nullable, active, created_at. LeaveType: id UUID, code unique, name, description, paid, balance_limited, allow_half_day, active. LeaveBalance: id UUID, employee_id, leave_type_id, period_year, entitled_days, carry_forward_days, adjustment_days, used_days or derived usage, created_at, updated_at. LeaveRequest: id UUID, reference unique (e.g. LV-000001), employee_id, leave_type_id, start_date, end_date, requested_days calculated by server, reason, status, submitted_at, decided_at, decided_by, decision_comment, version. LeaveRequestEvent: id UUID, request_id, actor_id, event_type, old_values/new_values JSONB, reason, occurred_at. Notification: id UUID, recipient_id, request_id nullable, event_id nullable, summary, created_at, read_at. IdempotencyRecord: user_id, operation, key, request_hash, response_status, response_body JSONB, expires_at.

Validation rules

Request reason: required, 10–2,000 trimmed characters (proposed). Leave type must be active and available to the employee. Dates must be valid ISO dates and in allowed request window. start_date ≤ end_date. requested_days is server-calculated and cannot be supplied as authoritative input. status, employee_id, decided_by and balance totals are server-controlled. Unknown JSON fields are rejected. UUIDs are used for internal identifiers; human-readable references are unique and never reused.

PostgreSQL constraints and indexes

Use foreign keys for user, leave type and request relationships; unique constraints for leave type code, request reference and idempotency scope; CHECK constraints for date ordering, nonnegative policy values and valid enums where appropriate. Index employee_id, status, start_date, end_date, manager_id and created_at for common queries. Use migrations (e.g. Alembic) for versioned schema changes. Use NUMERIC/DECIMAL or integer units for day balances; avoid floating-point arithmetic for leave balances.

7. REST API contract

Conventions

Base path: /api/v1. JSON request/response bodies. Protected endpoints require authentication. Lists return {items, page, page_size, total}. Default page=1 and page_size=20; maximum page_size=100. Use ISO 8601 dates/timestamps and UUIDs. Document all endpoints in FastAPI OpenAPI/Swagger.

Endpoints

POST /leave-requests — create request for authenticated employee; returns 201. GET /leave-requests — employee sees own requests; manager/HR scope is role-checked; supports status, leave_type_id, date range and pagination. GET /leave-requests/{id} — return authorized request or 404. POST /leave-requests/{id}/cancel — cancel an eligible Pending request; returns updated request. POST /leave-requests/{id}/approve — manager decision; may require version/If-Match; returns updated request. POST /leave-requests/{id}/reject — requires decision_comment; returns updated request. GET /leave-balances/me — current employee balances. GET /employees/{id}/leave-balances — HR/authorized manager access as defined by policy. GET /leave-types — list active leave types. POST /leave-types and PATCH /leave-types/{id} — HR Admin only. POST /employees/{id}/leave-balances/adjustments — HR adjustment with reason. GET /notifications and PATCH /notifications/{id} — current user's notifications. GET /leave-requests/{id}/history — authorized chronological audit history.

Example create request

Request: {"leave_type_id":"<uuid>","start_date":"2026-11-12","end_date":"2026-11-13","reason":"Personal commitments"}. The server determines employee identity, status, requested working days, reference and timestamps. A successful response returns the saved request with status Pending.

Example decision

POST /api/v1/leave-requests/{id}/approve with {"comment":"Coverage confirmed","version":1} (proposed body contract). The server checks manager scope, request state, policy and balance before committing the approval. Exact version handling may instead use an If-Match header; select one convention and apply consistently.

API summary table

Method	Path	Purpose
POST	/api/v1/leave-requests	Create leave request
GET	/api/v1/leave-requests	List authorized requests
GET	/api/v1/leave-requests/{id}	Read request details
POST	/api/v1/leave-requests/{id}/approve	Approve as authorized manager
POST	/api/v1/leave-requests/{id}/reject	Reject with comment
POST	/api/v1/leave-requests/{id}/cancel	Cancel eligible request
GET	/api/v1/leave-balances/me	Read own balances
GET/POST/PATCH	/api/v1/leave-types	Read/manage leave types; HR writes
GET/PATCH	/api/v1/notifications	Read/mark own notifications

8. Errors, notifications and reliability

Error contract

Use a consistent JSON shape such as {"error":{"code":"VALIDATION_ERROR","message":"Please correct the highlighted fields.","fields":{"end_date":"End date must be on or after start date."},"request_id":"..."}}. Proposed mappings: 400 MALFORMED_JSON; 401 UNAUTHENTICATED; 403 FORBIDDEN; 404 NOT_FOUND; 409 INVALID_STATE or OVERLAPPING_LEAVE or IDEMPOTENCY_CONFLICT; 412 VERSION_CONFLICT if conditional updates are used; 422 VALIDATION_ERROR; 500 INTERNAL_ERROR; 503 SERVICE_UNAVAILABLE. Never expose stack traces, secrets or SQL details.

Notification rules

On submission, notify the employee and the assigned manager/inbox as appropriate. On approval/rejection, notify the employee. On cancellation, notify the manager if action or awareness is required. HR balance adjustments may notify the affected employee according to policy. Create notifications in the same transaction as the business event. Notifications are personal; a user cannot read or mark another user's notification as read.

Reliability and security

Use PostgreSQL transactions for request decisions, balance updates, audit events and notifications. Parameterize queries or use SQLAlchemy ORM. Validate identity and authorization on every API request. Escape user-entered text when rendered. Apply request size limits, CORS allowlists, secret management and HTTPS in production. Log request ID, route, status and duration, but do not log sensitive leave reasons or authentication tokens. Use idempotency keys for create/cancel operations where retry duplication is a concern. Define backups and test restoration before production.

9. Quality and operational requirements

Quality targets

NF01 Security: all protected endpoints enforce authentication and role/ownership rules. NF02 Performance: proposed p95 response time ≤1 second for reads and ≤2 seconds for writes under a documented test load; confirm target and environment with stakeholders. NF03 Usability: responsive UI, keyboard operability and accessible validation messages. NF04 Data integrity: no duplicate or partially committed approval, balance or audit updates. NF05 Durability: successful writes survive application restart; daily backups and a tested restore process. NF06 Observability: structured logs and request IDs; no sensitive text or tokens in logs. NF07 Maintainability: typed Pydantic request/response models, service/repository separation, database migrations and API documentation. NF08 Testing: automated unit, API integration, authorization, lifecycle and React UI tests.

Architecture

React JS browser UI → FastAPI REST API → PostgreSQL. FastAPI owns validation, authorization, leave-day calculation, workflow, notifications and transactions. PostgreSQL is the system of record. Use SQLAlchemy 2.x and Alembic as proposed implementation choices; use Pydantic models for API schemas. Authentication may integrate with an organizational identity provider. These are implementation recommendations, not existing infrastructure claims.

Deployment and data handling

Maintain separate development, test and production configurations. Store credentials in environment-specific secret storage and never commit .env files. Use versioned migrations and a documented rollback procedure. Restrict database privileges to the application's needs. Define data retention, employee offboarding, backup access and recovery ownership before launch.

10. Acceptance criteria and test cases

Test setup

Use Employee A, Employee B, Manager A (assigned to Employee A), an unrelated Manager B, HR Admin and an inactive user. Seed active leave types and known balances. Verify HTTP response, visible UI and PostgreSQL persisted state.

TC01 — Valid submission

Given an authenticated employee and valid dates/type/reason, when they submit, then API returns 201, request has unique reference and Pending status, employee identity comes from the session, and audit/notification records are created.

TC02 — Required fields

Submit missing leave type, blank reason or missing dates; expect 422 field errors and no request/audit/notification rows.

TC03 — Invalid date range

Submit end date before start date; expect 422 and no persisted request.

TC04 — Unauthorized access

Call a protected endpoint without authentication or as an inactive user; expect 401 and no mutation.

TC05 — Ownership boundary

Employee A reads Employee B's request or balance; expect 404 or policy-defined denial without data leakage.

TC06 — Manager scope

Manager B attempts to approve Employee A's request; expect 403/404 per chosen disclosure policy and no state change.

TC07 — Approval flow

Manager A approves a valid Pending request; expect Approved status, decision actor/time, balance update and notification in one transaction.

TC08 — Rejection validation

Reject without a required comment; expect 422 and unchanged request.

TC09 — Insufficient balance

Attempt approval when balance is insufficient; expect conflict/validation error and no partial balance or status update.

TC10 — Overlap

Submit a request overlapping an existing Pending/Approved request; expect rejection according to the configured overlap policy.

TC11 — Duplicate decision

Two managers submit decisions concurrently; only one succeeds and the other receives a conflict response.

TC12 — Cancellation

Employee cancels an eligible Pending request; expect Cancelled status and audit event. Attempt cancellation of an ineligible Approved request; expect rejection.

TC13 — HR adjustment

HR adjusts an employee balance with a reason; balance and audit history update. Employee cannot call the same endpoint.

TC14 — Notification privacy

A user attempts to read or mark another user's notification as read; expect 404 and unchanged read state.

TC15 — XSS-safe rendering

Enter HTML/script-like text in reason or comment; it renders as inert text and never executes.

TC16 — Retry safety

Retry a creation request with the same idempotency key and same body; no duplicate request. Reuse the key with a different body; expect conflict.

TC17 — Transaction rollback

Force a database failure during approval notification/audit write; request status and balance remain unchanged.

TC18 — UI usability

Complete employee and manager journeys using keyboard only and at narrow and desktop widths; labels, focus, errors and actions remain usable.

TC19 — Persistence

Create and approve a request, restart the API, and confirm the committed request, balance and history remain available.

TC20 — API documentation

Verify OpenAPI schema lists request models, response models, authentication expectations and error responses.

Release acceptance

All core requirements and adopted proposed requirements map to passing tests. The employee request-to-decision journey and manager decision journey pass end to end. No unresolved defect may allow unauthorized access, incorrect balance changes, invalid decisions or lost committed data. Record test environment, build identifier, execution date and evidence.

Test coverage matrix

Area	Tests
Validation	TC02–TC03, TC08–TC10
Authorization	TC04–TC06, TC13–TC14
Workflow and balances	TC07–TC12
Reliability/security	TC15–TC17, TC19
Usability/documentation	TC18, TC20

11. Delivery plan and decisions

Implementation sequence

Task 1: confirm leave policies, identity model and roles; create React/FastAPI/PostgreSQL skeleton, migrations and test accounts. Task 2: implement authentication, role enforcement, leave types and employee balance view. Task 3: implement request submission, server-side date calculation, overlap checks, request list and detail screens. Task 4: implement manager inbox, approval/rejection, transaction-safe balance changes and audit history. Task 5: implement cancellation, notifications, HR adjustments and admin screens. Task 6: complete accessibility, error states, security tests, API/UI automation, backup/restore validation and deployment documentation.

Stakeholder decisions required

D01 Leave types and entitlement rules (annual, sick, casual, unpaid, etc.). D02 Accrual, carry-forward, expiry and negative balance policy. D03 Whether weekends and public holidays count; organization timezone and workweek. D04 Half-day/hourly leave support. D05 Who approves: direct manager, multi-level chain, HR or delegation. D06 Whether managers may approve their own requests. D07 Cancellation rules for approved leave and retroactive requests. D08 Whether pending requests reserve balance. D09 Notification channels and message content. D10 Identity provider, employee/manager data source, retention and production recovery targets.

Specification change control

Keep this specification with the source code. For each requirement change, update requirement IDs, data/API contracts and tests before changing implementation. Record version, date, rationale and impacted tests. AI-generated code must be reviewed against this specification; prompts do not override the approved contract.

Definition of ready

The document is an implementable draft for an initial release. It is not evidence that the system has been built or tested, and it does not assert approval of organization-specific leave policy. Production readiness requires stakeholder sign-off on the decisions above, a named operational owner and passing acceptance evidence.